

第九章 软件实现

本章速览

核心一句话：软件实现 = 把**详细设计**翻译成**源代码 + 可执行程序**的过程；狭义上等价于**"程序编码 + 单元测试"**，本质要求是**"可读性优先于效率"**。

核心概念（必背 6 条）

1. **广义软件实现：**详细设计 + 程序编码 + 单元测试 + 集成测试。
2. **狭义软件实现：**程序编码 + 单元测试（本章重点 = 程序编码）。
3. **软件实现任务四步：**选语言 → 选 IDE → 设计算法 → 编码实现。
4. **选语言的 6 个考量：**应用领域、用户要求、现有工具、成本、程序员水平、可移植性。
5. **两大程序设计方法：**结构化（自顶向下、逐步求精）、面向对象（类为核心）。
6. **程序设计风格 5 大方面：**源程序文档化、数据说明、语句结构、输入/输出、错误处理。

核心问题（高频考法）

- 广义 vs 狭义软件实现的差别在哪？ **判断/简答**
- 选程序设计语言要考虑哪些方面？ **简答**
- 结构化程序设计三种基本结构是什么？ **选择**
- 序言性注释 vs 功能性注释——位置、目的、内容差别？ **选择/填空**
- 标识符命名规范——下列哪个最规范？ **选择**
- "源程序文档化"包括哪三方面？ **填空/简答**
- 良好的程序设计风格表现在哪些方面？ **简答**

高频考点（PPT 主导）

- ★ **序言性注释 vs 功能性注释**（2010、2023 真题，必考）
- ★ **标识符命名通用规范**（2018 真题，4 选 1）
- ★ **源程序文档化 = 标识符命名 + 程序注释 + 源程序布局**（2023 填空）
- ★ **结构化程序设计三种基本结构**（顺序、选择、循环/重复，2010 真题）
- ★ **选择程序设计语言的 6 个考量因素**
- ★ **良好的程序设计风格 5 大方面**

题目关键词导航

关键词	对应知识点	题型
序言性注释	源程序文档化·程序注释	选择/填空/简答
功能性注释	源程序文档化·程序注释	选择/简答
标识符/命名规范	源程序文档化·标识符命名	选择
源程序文档化	程序设计风格 5 大方面之一	填空/简答
软件实现（包含什么）	9.1 实现的目标与任务	判断
程序设计语言选择	9.2.1 6 个考量因素	简答
结构化程序设计	9.3.1 自顶向下逐步求精	选择/简答
三种基本控制结构	顺序、选择、循环	选择
面向对象程序设计	9.3.2 类的实现 + 应用程序实现	简答
良好的程序设计风格	9.4 5 个方面	简答
清晰第一 / 效率第二	9.4.3 语句结构原则	判断/简答
数据说明次序	9.4.2 规范化顺序	简答
错误处理	9.4.5 5 种处理方式	简答
输入/输出方法	9.4.4 9 条原则	简答
IDE / 集成开发环境	9.2.2 组成与功能	简答/填空

知识主线

本章逻辑链（按"为什么实现 → 用什么实现 → 怎么实现 → 怎么实现得好"递进）：

1. **为什么实现**（9.1）——目标是把《详细设计说明书》翻译成源程序、可执行程序、程序说明书；任务四步：选语言→选 IDE→算法设计→编码实现。
2. **用什么实现**（9.2）——程序设计语言（按 6 个维度选）+ 集成开发环境 IDE（编辑器 + 编译器 + 调试器 + ...）。
3. **怎么实现**（9.3）——两种程序设计方法：结构化（自顶向下、逐步求精；顺序/选择/循环三基本结构）与面向对象（以类为核心，含类的实现与应用程序实现）。
4. **怎么实现得好**（9.4）——程序设计风格 5 大方面：**源程序文档化**（标识符命名 + 程序注释 + 程序布局）、**数据说明**、**语句结构**（清晰第一效率第二）、**输入/输出**、**错误处理**。

考点密度：9.4（程序设计风格）> 9.3（程序设计方法）> 9.2（语言与 IDE）> 9.1（概述）。9.4.1（源程序文档化）几乎是每年必出的 1 道 1-2 分小题。

核心知识点详解（PPT 主导）

9.1 软件实现的目标与任务 必考概念

广义 vs 狭义软件实现 (教材 9.1 原文):

- 广义软件实现: 包括**详细设计、程序编码、单元测试、集成测试**。
 - 狭义软件实现: 指**程序编码 + 单元测试**。本章聚焦程序编码。
- 软件实现的目标: 选择某种程序设计语言, 将**详细设计结果**进行编码实现, 形成**可执行的软件系统**。

输入 / 输出:

- 输入: **《详细设计说明书》**。
- 输出: 源程序代码、可执行程序、程序说明书。

软件实现的任务 (4 步):

1. **程序设计语言选择**: 根据系统特点和设计方案, 选择一种或多种语言。
2. **集成开发环境选择**: 选成熟、稳定、易用的 IDE; IDE 非必需, 但能极大提高开发效率。
3. **程序实现算法设计**: 针对特定功能模块, 设计所需数据结构与算法。
4. **程序编码实现**: 在 IDE 中按算法和数据结构编码、调试、输出可执行程序。

★考点 广义/狭义软件实现都可能考**判断题**。“软件实现包含单元测试”无论广义还是狭义都成立 (2010-2011 判断第 8 题考过)。

9.2 程序设计语言与 IDE

9.2.1 选择程序设计语言的 **6 个考量因素** 高频简答

1. **软件的应用领域** (首要因素): 不同领域语言效率差异大。
 - 商业 → Cobol、C++、Java
 - 科学计算 → Fortran、C、C++、Java
 - 系统级 → C、C++、Java
 - 人工智能 → LISP、Prolog
 - Web/网络应用 → HTML、JavaScript、JSP、PHP、C#
2. **系统用户的要求**: 交付后由用户维护时, 用户常指定熟悉的语言。
3. **现有的工具环境**: 已有开发工具/编译环境能否满足; 可省配置新工具的开销。
4. **开发环境成本**: 必须重新配置时考虑软件成本; 可选免费开源工具。
5. **程序员水平**: 选程序员熟悉的语言, 节省时间、提高质量。
6. **软件可移植性的要求**: 跨平台/跨操作系统须选**标准化程度高、可移植性好的语言**。

■ **记忆** 口诀: “应、用、工、成、员、移” (应用领域、用户要求、工具环境、成本、程序员水平、可移植性)。

9.2.2 集成开发环境 (IDE) 简答

定义: IDE (Integrated Development Environment) 是运行在操作系统中的**图形界面软件系统**, 将**编辑源程序、调试程序、生成可执行文件**等功能集成到一起。

最小组成: 编辑器 + 编译器工具链 + 调试器。

常用 IDE 的 7 大功能:

1. 项目和源代码的管理功能。
2. 源代码编辑提示功能 (自动补全)。
3. 文本编辑功能 (复制、粘贴、查找、替换)。
4. 程序跟踪调试功能 (断点、单步)。
5. 生成可执行文件功能。
6. 与其他插件结合的功能。
7. 屏幕管理功能。

常见 IDE: Visual Studio、Eclipse、NetBeans、Delphi、C++ Builder、JBuilder。UNIX/Linux 下多用 vi + gcc + make + makefile。

9.3 程序设计方法 必考

衡量程序质量的主要标准 (在保证正确的前提下): **可阅读性、可理解性、可修改性、可维护性、可扩展性**。两大方法是**结构化程序设计**与**面向对象程序设计**。

9.3.1 结构化程序设计方法

基本思想: “**自顶向下, 逐步求精**”——将程序按功能划分为若干基本模块, 形成**树状结构**, 每个模块内部均由**顺序、选择、循环三种基本结构**组成。

四条基本原则:

1. 使用语言中的**顺序、分支、循环**等有限的基本控制结构表示程序逻辑。
2. 选用的控制结构只准许有一个**入口**和一个**出口**。
3. 复杂结构应该用基本控制结构进行**组合嵌套**来实现。
4. 语言中没有的控制结构可用一段**等价的程序段**模拟, 但要求该程序段在整个系统中应**前后一致**。

★考点 三种基本结构: **顺序、选择 (分支)、循环 (重复)**——2010 真题单选 5 题干“结构化程序设计采用的三种基本控制结构是”选 D (顺序、选择、重复)。注意“重复”即“循环”, 两个词都可能出现。

“**自顶向下, 逐步求精**”举例 (教材例: 用筛选法求 100 以内素数):

1. 第 1 层: 写出框架 建立 2~100 数组 → 建立 2~10 素数表 → 剔除倍数 → 输出剩余。
2. 第 2 层: 把每一步细化成循环语句。
3. 第 3 层: 把循环体进一步细化为能直接翻译成 C 语言的语句。

■ **图示 结构化程序设计方法 (图 9-1)**——核心表达: 整个程序自顶向下被分解成**树形结构**。根节点=主问题 (总程序); 第 1 层=按功能划分的若干相对独立的小问题 (模块); 每个模块再向下细化成更小的子问题; 同一层的节点之间相互独立, 任何一步出错只影响其下层节点, 不影响同层兄弟节点; 最底层=可直接用基本控制结构 (顺序、选择、循环) 实现的语句。

9.3.2 面向对象程序设计方法 (OOP)

基本思想: 对问题空间进行**自然分割**, 以更接近人类思维的方式建立问题域模型。引入 **对象** 代表事物, **消息** 建立事物间联系, **类和继承** 描述对象。

核心问题: **类的实现**。所有数据和操作都被封装在类中, 整个应用封装在更高级的类中。

面向对象程序设计 = 类的实现 + 应用程序实现。

1. **类的关系**: 关联、聚合/组合、继承、依赖 (详见 4.4 节)。

2. **类的实现** (分层开发方案, 须考虑 7 项内容):

1. **软件库 (Software Base / 类库)**: 系统库或用户库, 存储可复用代码。
2. **复用 (Reuse)**: 先考虑类的复用; 高层类常设计为**抽象类**。两种复用方式:
 - (a) “**原封不动**”地复用: 所需类已存在, 建立实例即可;
 - (b) “**进化性**”复用: 通过继承现存类, 渐增地设计新类。
3. **断言 (Assertions)**: 把参数约束、循环约束编入代码; 分为**先决条件** (操作执行前必须满足的条件) 和**后置条件** (操作执行后必须满足的条件)。C/C++ 用 assert.h。
4. **调试 (Debugging)**: 数据封装在对象内部, 受限于封装; 可借助交互式调试工具。
5. **内建错误处理 (Built-In Error Handling)**: 利用语言提供的异常处理机制。
6. **用户定义的错误处理 (User-Defined Error Handling)**: 全局错误处理器对象 / 每个类定义 error 操作。
7. **多重实现 (Multiple Implementation)**: 同一个类可以多种方式实现。

3. **应用程序的实现**: 在所有类被实现之后进行。建立类的实例, 通过消息实现对象之间的通信和操作。纯面向对象语言**没有主过程** (常常是交互的)。

9.4 程序设计风格 本章最重要

定义：程序设计风格=设计、编制、调试程序的方法和过程。其标准是**使程序易于阅读和理解**。

程序设计风格一般表现在 5 个方面：

1. **源程序文档化**
2. **数据说明的方法**
3. **语句结构**
4. **输入/输出方法**
5. **错误处理**

★考点 "良好的程序设计风格表现在哪些方面"几乎每两年出一道简答——答 5 个方面足矣。

9.4.1 源程序文档化 最高频

定义：源程序文档化=程序源代码本身具有良好的**可读性**，如同进行代码文档说明。

三方面：**标识符命名 + 程序注释 + 源程序布局**。

(1) **标识符命名** (5 条通用注意事项)：

1. 能反映含义的英文全称，不使用无意义字符串。如 firstName、provideTime、mailAddress；x1、fn、y2 不建议。
2. 尽量使用专业术语。如 CRM 系统中"客户"用 customer，不用 client 或 user 避免歧义。
3. 注意大小写的含义（编程规范约定）。如 Java：包全小写、类/接口首字母大写、变量首字母小写（驼峰）、Static Final 全大写。
4. 标识符不宜过长，必要时用缩写但规则要一致并加注释。
5. 避免使用只有略微差异的标识符（如 SqlServer vs SQLServer；number vs nomber；student vs students）。

★考点 2018–2019 单选 10："下面哪个标识符符合规范化命名的要求"

- A. abc123 (无意义) B. mailAddress (驼峰、有意义) D. thisstudentmanagementclass (过长、无大小写区分) E. Static (关键字/全大写不规范)

标准答案：B。判断口诀："有意义 + 长度适中 + 遵循大小写规范"。

(2) **程序注释**——沟通程序员与日后读者的关键手段。正规程序文本中注释行占**1/3 到 1/2 甚至更多**。分两类：

- **序言性注释** (详见下方 9.4.1.专)
- **功能性注释** (详见下方 9.4.1.专)

注释可通过 Javadoc 等工具自动生成程序说明文档。

(3) **源程序布局**——利用**空格、空行、移行**提高可视化。良好风格 8 条：

1. 一行最好只书写一个语句。
2. 每行缩进**两个字符**，不使用 Tab 键（不同工具宽度不同）。
3. 选择/循环语句中的程序段**向右阶梯移行**，使逻辑结构清晰。
4. 程序说明部分和执行部分之间、不同功能模块之间用**空行**分隔；避免每行之间都空行。
5. 不随意省略可省略的**大括号**，大括号独自占一行。
6. 结束括号与对应的开始括号**缩进量相同**。
7. 不随意使用无意义的小括号。
8. 小括号：左括号与后一个字符之间、右括号与前一个字符之间**不应出现空格**。

★ 序言性注释 vs 功能性注释 (必考对比)

序言性注释 (PPT+教材 9.4.1)：

- 位置：每个程序模块的开头部分 (**程序代码之前**)。
- 目的：解释程序的主要内容与难点；给出**程序的整体说明**，对理解程序本身具有引导作用。
- 内容 (7 项)：
 1. 程序标题 (模块名称)
 2. 模块功能和目的说明 (模块描述)
 3. 主要算法 (可选, option)
 4. 接口说明 (与其他模块的调用关系)
 5. 有关数据描述
 6. 模块位置
 7. 开发简历：创建者/创建时间；修改者/修改时间/修改内容；版本

功能性注释 (PPT+教材 9.4.1)：

- 位置：**嵌在源程序体中** (局部、紧邻被解释的语句或程序段)。
- 目的：对程序体中复杂、难于理解的程序结构进行局部说明；**描述一段程序** (必要时也可对某一段进行说明)，**不具体解释"如何实现"**。
- 书写注意：
 - 用于描述一段程序，而不是每一个语句；
 - 用缩进和空行使程序与注释容易区别；
 - 保持注释与代码的一致性——**修改代码的同时，对应的功能性注释也要进行修改**。

★考点 2009–2010 单选 6："序言性注释内容不应有"

- A. 模块的功能 B. **语句的功能** C. 模块的接口 D. 开发历史
标准答案：B。

关键理由：序言性注释是模块级整体说明，"**语句的功能**"属于**功能性注释**的范畴 (且功能性注释描述的是"一段程序"而非"每一个语句")。

2023–2024 填空 6："源程序文档化主要体现在为代码进行 (**序言性注释**) 和功能性注释。"

9.4.2 数据说明 (Data Declaration)

声明数据结构的过程称作**数据说明**。须遵循 3 条风格：

1. **数据说明的次序应当规范** (便于查找、测试、维护)。例如，类型说明顺序：
常量说明 → 复杂类型数据结构 → 简单变量类型说明 (顺序：整型 → 实型 → 字符型 → 逻辑型) → **数组说明**。
2. **多个变量名用一个语句说明时**，应当对这些变量按字母顺序排列。
3. **复杂的数据结构**，应当使用注释来说明在程序实现时这个数据结构的用途和特点。

9.4.3 语句结构 (Statement Structure) 高频原则

核心原则：语句结构力求简单、直接，**不能为了片面追求效率而使语句复杂化**。21 条原则中重点掌握：

- 1. 一行内只写一条语句，并采取适当的移行格式。
- 2. 程序编写**首先应当考虑清晰性**，不刻意追求技巧。
- 3. 程序编写得简单，直截了当地说明程序员的用意。
- 4. **除非对效率有特殊要求，否则程序编写要做到“清晰第一，效率第二”**。
- 5. 首先保证程序**正确**，然后才考虑**执行速度**。
- 6. 避免使用临时变量使可读性下降。
- 7. 使用**公共函数**代替重复使用的表达式。
- 8. 使用**括号**清晰地表达算术/逻辑表达式的运算顺序。
- 9. 尽量只采用基本的控制结构编写程序。
- 10. 用**逻辑表达式**代替分支嵌套。
- 11. 避免使用空的 ELSE 语句和 IF...THEN IF... 语句。
- 12. 使与"判定"相联系的动作尽可能紧跟"判定"语句。
- 13. 避免采用过于复杂的条件测试；避免过多的循环嵌套和条件嵌套。
- 14. 使用数组以避免重复的控制序列。
- 15. 尽可能用通俗易懂的**伪码**描述程序流程，再翻译成必须使用的语言。
- 16. 使模块功能尽可能**单一**，模块间的耦合清晰可见。
- 17. 利用**信息隐蔽**，确保每一个模块的独立性。
- 18. 从数据结构出发构造程序。
- 19. 不修补不好的程序，应重新编写。
- 20. 对于太大的程序，应**分块编写、测试，然后再集成**。

面向对象程序设计补充 6 条：

- 1. 类的方法不宜太大，每个类的方法**代码行不超过 50 行**为最佳。
- 2. 减少参数个数。
- 3. 避免类的继承层数过多。
- 4. 尽量少用运算符重载。
- 5. 尽最大可能使用错误处理过程，并对状态和错误进行记录。
- 6. 充分考虑异常的处理。

★考点 “清晰第一，效率第二”——可作为判断题的核心陈述：“**程序设计应优先考虑效率而非清晰性**”= 错。

补充知识点（教材独有 / 非PPT主导）

程序设计方法学的内容

程序设计方法学是讨论程序的性质、程序设计理论和方法的科学，包含：

- 结构化程序设计
- 面向对象程序设计
- 程序正确性证明
- 程序变换
- 程序的形式说明与推导
- 程序综合
- 自动程序设计

普通程序员最需掌握的是**结构化**和**面向对象**两种。

断言 (Assertions) 深入

主动实现类的方法是把来自类的设计信息直接纳入代码，特别要求把**参数约束、循环执行**等编入到代码中——通过表示断言的语言机制实现。

- **先决条件** (Pre-condition)：陈述操作执行**之前**必须满足的条件。
- **后置条件** (Post-condition)：陈述操作执行**之后**必须满足的条件。

C/C++ 通过 assert.h 支持断言。例如对 pop 操作断言“栈是否非空”，失败时打印源文件名和行号。

核心对比表

9.4.4 输入和输出 (I/O)

核心：输入/输出方式和格式应当尽可能**方便用户使用**。9 条原则：

- 1. 对所有输入数据进行**检验**，识别错误输入，保证有效性。
- 2. 检查输入项各种重要组合的**合理性**，必要时报告输入状态信息。
- 3. 输入步骤和操作尽可能**简单**，保持简单的输入格式。
- 4. 允许使用**自由格式**输入。
- 5. 应允许**默认值**。
- 6. 输入一批数据时，最好使用**输入结束标志**，而不要由用户指定输入数据数目。
- 7. 交互式输入/输出时，屏幕上使用**提示符**明确提示请求，指明可使用选择项的种类和取值范围；输入过程中和结束时给出状态信息。
- 8. 当程序设计语言对输入/输出格式有严格要求时，应保持输入格式与输入语句的要求**一致**。
- 9. 给所有的输出**加注解**，并设计输出报表格式。

9.4.5 错误处理 (Error Handling)

系统的错误处理能力影响软件的**正确性、稳定性**及**非功能性属性**。两类技术：

- **避开错误技术**：在开发过程中不让差错潜入软件。
- **容错技术**：对某些无法避开的差错，使其影响减至最小。

5 种错误处理方法：

- 1. **返回错误代码**：调用者根据错误代码处理。
- 2. **调用错误处理函数**：统一处理，便于集中管理。
- 3. **显示错误信息**：提示用户（如非法数据时给出正确格式提示）。
- 4. **记录日志**：写日志文件后继续执行。
- 5. **退出程序**：对安全性要求较高的程序适用，防止继续执行带来破坏。

两种用户定义错误处理技术

- 1. **全局错误处理器对象**：每个类都能存取全局对象；用户对象检测出错误时，发送一个消息给全局对象（消息包含出错信息字符串和错误严重程度整数）。
- 2. **每类自定义 error 操作**：每个类定义或再定义一个命名为 error 的操作；它不是类的共有界面，而是隐蔽的实现部分，可被公共操作调用以检测错误。

两种技术都不允许嵌套的错误处理。

注释行数量标准

在正规的程序文本中，注释行的数量占整个源程序的**1/3 到 1/2，甚至更多**。这是判断“是否过度注释”的隐性参考线。

UV/TDD 等过程模型对应的 IDE 实践

UNIX/Linux 下 IDE 支持不如 Windows，常采用 vi + gcc + make + makefile 完成大型复杂系统编译。Windows 下采用 IDE（如 Visual Studio）已成主流。

软件复用的两种方式

面向对象实现类时，优先考虑复用。最高层次的类设计为**抽象类**，定义大部分功能但缺少实现，由具体类提供实现。

- 1. **"原封不动"地复用**：所需类已存在，建立它的实例即可。
- 2. **"进化性"复用**：不存在完全符合要求的类，但有类似功能的类存在——通过**继承**现存类渐增地设计新类。

表 1 广义软件实现 vs 狭义软件实现

维度	广义软件实现	狭义软件实现
范围	详细设计 + 程序编码 + 单元测试 + 集成测试	程序编码 + 单元测试
起点	从详细设计开始	从编码开始
终点	包含集成测试（接近交付）	止于单元测试
本章范围	——	本章特指
共同点	无论广义还是狭义，都包含单元测试	

表 2 序言性注释 vs 功能性注释（必考）

维度	序言性注释	功能性注释
位置	每个程序模块的开头部分（程序代码之前）	嵌在源程序体中（局部）
目的	给出程序整体说明，引导读者理解程序	对复杂难懂的程序结构进行局部说明
粒度	模块级（整体）	程序段级（一段或必要时某一语句段）
是否含语句级解释	否（"语句的功能"不属于其内容）	是（但描述"一段"而非"每一个语句"）
典型内容	标题、功能/目的、主要算法、接口说明、数据描述、模块位置、开发简历	程序段的功能/编写目的（不解释"如何实现"）
是否包含修改记录	是（开发简历含修改者/时间/版本）	否
修改要求	开发简历随修改更新	修改代码同时修改注释

表 3 结构化 vs 面向对象程序设计

维度	结构化程序设计（SP）	面向对象程序设计（OOP）
提出时间	20 世纪 60 年代末—70 年代初	随 OOP 语言成熟发展而来
基本思想	自顶向下，逐步求精；按功能划分模块	自然分割问题空间；以对象/类为核心
程序结构	树状结构，功能相对独立	类与对象之间通过消息通信
基本结构	顺序、选择、循环	封装、继承、多态
核心关注	模块功能与流程控制	类的实现（数据 + 操作封装）
复用方式	子程序调用	类库 + 继承 + 复用
典型代表	C、Pascal	Java、C++、C#

表 4 标识符命名"该做"与"不该做"

类型	建议示例	反例（不建议）
有意义英文全称	firstName / mailAddress	x1 / fn / abc123
专业术语	CRM 用 customer	用 client/user 表"客户"
大小写规范	Java 类名首字母大写：OrderService	Static、SQLServer
长度适中	mailAddress	thisstudentmanagementclass
避免轻微差异	不同含义用不同根词	number vs nomber

表 5 错误处理 5 种方法对比

方法	典型场景	优缺点
返回错误代码	调用方需根据码做不同处理	简单，但易被忽略
调用错误处理函数	集中错误管理	统一，但耦合度高
显示错误信息	用户输入非法数据	友好，但仅面向人
记录日志	系统继续运行	可追溯，但需配套日志系统
退出程序	安全性要求高	避免破坏，但用户体验差

开放题答题模板

Q1：什么是源程序文档化？它包括哪几个方面？

定义：源程序文档化指**程序源代码本身具有良好的可读性**，如同进行代码文档说明。从软件工程角度，源程序除质量要求外，为便于后期维护和修改，必须从提高可读性出发达到源程序文档化。

三方面：

1. **标识符命名**（模块名/类名/方法名/变量名/常量名）——名称清楚表示含义；用专业术语；注意大小写与长度；添加前缀/后缀；避免轻微差异。
2. **程序注释**——分**序言性注释**（位于模块开头，含标题、功能、算法、接口、开发简历）和**功能性注释**（嵌在程序体中，描述一段程序）。
3. **源程序布局**——统一编码规范，规定合理的缩进、空格、空行、移行；每行缩进两字符；一行一语句；大括号独占一行。

Q2：良好的程序设计风格表现在哪些方面？

5 个方面（必背）：

1. **源程序文档化**（标识符命名 + 程序注释 + 程序布局）
2. **数据说明**（次序规范；多变量按字母顺序；复杂数据结构加注释）
3. **语句结构**（清晰第一、效率第二；一行一语句；避免过多嵌套；用括号明确运算顺序；模块功能单一）
4. **输入/输出方法**（输入检验、自由格式、默认值、输入结束标志；输出加注释）
5. **错误处理**（返回错误代码、调用错误处理函数、显示错误信息、记录日志、退出程序）

加分项：可补充"标准是使程序易于阅读和理解"。

Q3：选择程序设计语言时应考虑哪些方面？

6 个方面（口诀：应·用·工·成·员·移）：

1. **软件的应用领域**——不同领域语言效率和能力差异大，选该领域最常用的语言（如 Web → Java/C#；系统级 → C；数据库 → SQL）。
2. **系统用户的要求**——交付后由用户维护时，用户指定熟悉的语言。
3. **现有的工具环境**——已有开发工具/编译环境是否满足，避免重复配置。
4. **开发环境成本**——必须重新配置时考虑成本，可选免费开源工具。
5. **程序员的水平**——选程序员熟悉的语言，节省时间提高质量。
6. **软件可移植性的要求**——跨平台须选标准化程度高、可移植性好的语言。

Q4：序言性注释与功能性注释有何区别？

位置：序言性注释位于每个程序模块的开头；功能性注释嵌在源程序体中。

目的：序言性注释给出程序整体说明，具有引导作用；功能性注释对复杂程序结构进行局部说明。

内容：序言性注释包含标题、功能/目的、主要算法、接口说明、数据描述、模块位置、开发简历；功能性注释只描述一段程序的编写目的，不解释"如何实现"。

关键区别：是否含"语句的功能"——序言性注释不含；功能性注释属程序段级。

Q5：结构化程序设计方法的主要思想是什么？

核心思想：“**自顶向下，逐步求精**”。

1. 使用**顺序、选择、循环**三种基本控制结构表示程序逻辑。
2. 选用的控制结构只有一个入口和一个出口。
3. 复杂结构用基本控制结构**组合嵌套**实现。
4. 将程序按功能划分为若干**基本模块**，形成**树状结构**，功能相对独立。
5. 同一层节点细化工作相互独立，错误只影响下层节点。

Q6：面向对象程序设计方法的主要思想是什么？

基本思想：对问题空间进行**自然分割**，以接近人类思维的方式建立问题域模型。引入**对象**代表事物，用**消息**建立事物间联系，用**类和继承**描述对象。

核心问题：类的实现。所有数据和操作封装在类中，整个应用封装在更高级的类中。

主要内容：

1. **类的实现**：含软件库、复用（原封不动 / 进化性）、断言（先决条件 + 后置条件）、调试、内建错误处理、用户定义错误处理、多重实现。
2. **应用程序的实现**：所有类实现之后建立实例，通过消息通信。

目标：构造出**模块化的、可重用的、维护性好的软件**，限制软件复杂性、降低开发维护费用。

Q7：软件实现都包含哪些方面的工作？包含哪些具体任务？

包含方面（狭义）：程序编码 + 单元测试；广义还包括详细设计、集成测试。

具体任务（4步）：

1. 程序设计语言选择
2. 集成开发环境选择
3. 程序实现算法设计（含数据结构）
4. 程序编码实现（编辑、调试、生成可执行程序）

输入 / 输出：输入《详细设计说明书》；输出源程序代码、可执行程序、程序说明书。

Q8：什么是断言？分为哪两类？

定义：断言是一个语句，表达对一个过程、一个值、一段代码的**约束**。把来自类的设计信息（参数约束、循环约束等）直接纳入代码。

两类：

- **先决条件**：操作执行**之前**必须满足的条件。
- **后置条件**：操作执行**之后**必须满足的条件。

C/C++ 通过 `assert.h` 支持。条件测试失败时打印源文件名与行号。

历年真题汇总与详解

一、单项选择题

【2009–2010 期末 A · 单选 5】结构化程序设计采用的**三种基本控制结构**是（ ）

- A、顺序、分支、选择 B、选择、循环、重复 C、输入、变换、输出 D、**顺序、选择、重复**

标准答案：D

解析：教材 9.3.1 原文——“每个模块内部均由**顺序、选择和循环**三种基本结构组成”。“重复”即“循环”的同义表达；A 项把“分支”与“选择”重复列出，缺循环；B 项同样重复；C 项是输入–变换–输出（IPO）模型，非控制结构。

【2009–2010 期末 A · 单选 6】源程序文档化要求在每个模块之前加序言性注释。该注释内容不应有（ ）

- A、模块的功能 B、语句的功能 C、模块的接口 D、开发历史

标准答案：B

解析：教材 9.4.1 明确——**序言性注释**包含：程序标题、模块功能和目的说明、主要算法、接口说明、数据描述、模块位置、开发简历。“**语句的功能**”是功能性注释的范畴（且功能性注释描述的是“一段程序”而非“每一个语句”）。序言性注释是**模块级**整体说明，不涉及语句级解释。

【2018–2019 期末 B · 单选 10】下面哪个标识符符合规范化命名的要求：_____

- A. abc123 B. mailAddress D. thisstudentmanagementclass E. Static

标准答案：B

解析（按教材 9.4.1 五条命名规范逐一判断）：

- A abc123——无实际含义，违反“使用能反映含义的英文全称描述”。
- B mailAddress——有意义、驼峰命名、长度适中，**符合**。
- D thisstudentmanagementclass——过长，且全小写无单词分隔，违反“标识符不宜过长”和“注意大小写”。
- E Static——首字母大写、形似 Java 关键字 static，违反“避免使用只有略微差异的标识符”和编码规范。

判断口诀：有意义 + 长度适中 + 遵循大小写规范 + 不与关键字冲突。

【参考补充题】程序编写首先应当考虑（ ）

- A、执行效率 B、**清晰性** C、技巧性 D、简洁性

标准答案：B

解析：教材 9.4.3 第 2 条——“程序编写首先应当考虑**清晰性**，不刻意追求技巧”；第 4 条——“**除非对效率有特殊要求，否则程序编写要做到清晰第一，效率第二**”。

二、判断题

【2010–2011 期末 B · 判断 8】软件实现包含单元测试。（ ）

标准答案：√（正确）

解析：教材 9.1 原文——“广义的软件实现包括详细设计、程序编码、单元测试和集成测试。狭义上理解的软件实现指**程序编码和单元测试**”。**无论按广义还是狭义，软件实现都包含单元测试**。

【参考补充题】程序设计中，为了提高执行效率，可以牺牲程序的清晰性。（ ）

标准答案：×（错误）

解析：教材 9.4.3 第 4 条——“除非对效率有特殊的要求，否则程序编写要做到**清晰第一，效率第二**”。一般情况下，清晰性优先于效率；只有对效率有特殊要求时才考虑牺牲部分清晰性换取效率。

【参考补充题】在源程序中，使用 Tab 键进行缩进是一种良好的程序布局风格。（ ）

标准答案：×（错误）

解析：教材 9.4.1 程序布局第 2 条——“**每行缩进两个字符。不使用 Tab 键进行缩进**，因为不同的源代码管理工具中 Tab 键因用户设置不同而具有不同的宽度”。应使用空格代替 Tab。

【参考补充题】序言性注释位于程序模块的开头，功能性注释嵌在源程序体中。（ ）

标准答案：√（正确）

解析：教材 9.4.1 原文——序言性注释“通常置于每个程序模块的开头部分”；功能性注释“嵌在源程序体中”。位置差别是两类注释的核心区别之一。

三、填空题

【2023-2024 期末 A · 填空 6】源程序文档化主要体现在为代码进行（ ）和功能性注释。

标准答案：序言性注释

解析：教材 9.4.1——程序注释分为序言性注释（位于模块开头，给出整体说明）和功能性注释（嵌在源程序体中，描述一段程序的编写目的）。两者共同构成“程序注释”这一文档化手段。

【参考补充题】程序设计风格一般表现在源程序文档化、数据说明的方法、（ ）、输入/输出方法和错误处理 5 个方面。

标准答案：语句结构（或“表达式和语句结构”）

解析：教材 9.4——程序设计风格 5 大方面为：①源程序文档化、②数据说明的方法、③语句结构（或表达式和语句结构）、④输入/输出方法、⑤错误处理。

【参考补充题】软件实现的任务包括程序设计语言选择、集成开发环境选择、（ ）和程序编码实现。

标准答案：程序实现算法设计

解析：教材 9.1.2——软件实现的任务四步：①程序设计语言选择、②集成开发环境选择、③程序实现算法设计、④程序编码实现。

四、简答题

【参考补充题】请说明什么是源程序文档化？源程序文档化包括哪些方面？

定义：源程序文档化指程序源代码本身具有良好的可读性，如同进行代码文档说明。从软件工程角度，源程序除质量要求外，为便于后期维护和修改，必须从提高可读性出发达到源程序文档化。

包括三方面：

- 1. 标识符命名——模块名、变量名、常量名、子程序名、类等。命名应清楚表示含义；使用专业术语；注意大小写及长度；避免使用只有略微差异的标识符。
- 2. 程序注释——含序言性注释（程序标题、模块功能和目的、主要算法、接口说明、数据描述、模块位置、开发简历）与功能性注释（嵌在源程序体中，描述一段程序，不解释“如何实现”）。
- 3. 源程序布局——统一编码规范，规定合理的缩进、空格、空行、移行；一行一语句；每行缩进两字符；大括号独占一行。

【参考补充题】良好的程序设计风格表现在哪些方面？

5 个方面：

- 1. 源程序文档化（标识符命名、程序注释、源程序布局）。
- 2. 数据说明的方法（次序规范；多变量按字母顺序排列；复杂结构加注释说明用途和特点）。
- 3. 语句结构（清晰第一、效率第二；一行一语句；用括号明确运算顺序；避免过多嵌套；模块功能单一）。
- 4. 输入/输出方法（输入数据检验；输入步骤简单；允许自由格式和默认值；输出加注解和报表格式）。
- 5. 错误处理（返回错误代码、调用错误处理函数、显示错误信息、记录日志、退出程序）。

得分关键词：源程序文档化、数据说明、语句结构、输入/输出、错误处理。

【参考补充题】请简述结构化程序设计方法的基本思想。

基本思想：“自顶向下，逐步求精”。

- 1. 将程序按照功能划分为若干基本模块，这些模块形成一个树状结构，功能相对独立。
- 2. 每个模块内部均由顺序、选择、循环三种基本结构组成。
- 3. 模块实现过程中可以使用子程序。
- 4. 复杂结构应该用基本控制结构进行组合嵌套来实现。
- 5. 选用的控制结构只准许有一个入口和一个出口。

效果：提高了程序的执行效率，减少了程序出错的概率，极大减少了维护的费用。

【参考补充题】请简述选择程序设计语言时应考虑的因素。

6 个因素：

- 1. 软件的应用领域——针对应用领域选最常用语言。
- 2. 系统用户的要求——交付后由用户维护时常指定熟悉语言。
- 3. 现有的工具环境——已有工具和编译环境能否满足。
- 4. 开发环境成本——必须重新配置时考虑成本。
- 5. 程序员的水平——选程序员熟悉的语言。
- 6. 软件可移植性的要求——跨平台选标准化程度高、可移植性好的语言。

【2015-2016 期中 A · 简答 3】请解释“推迟了软件实现”是瀑布模型的一大优点。

含义：“推迟”特指软件编码（软件实现）之前执行了瀑布模型中的需求分析和软件设计等活动，提高了软件质量。

解析：本题属第二章瀑布模型知识点（教材 2.3.1）。瀑布模型强调在编码之前必须先完成需求分析和设计工作，避免开发人员急于编写代码而忽视分析与设计，最终导致代码频繁改动、结构混乱、质量下降。本章可联系“软件实现是详细设计的后续阶段，输入《详细设计说明书》”理解。

A4 双栏 Cheat Sheet（考点速查）

软件实现·定义

- 广义 = 详细设计 + 编码 + 单元测试 + 集成测试
- 狭义 = 编码 + 单元测试
- 本章 = 程序编码（输入《详细设计说明书》，输出源程序、可执行程序、程序说明书）
- 判断：软件实现包含单元测试 = √

实现任务 4 步

- 1. 选程序设计语言
- 2. 选 IDE
- 3. 设计算法与数据结构
- 4. 编码实现

选语言 6 因素

口诀：应·用·工·成·员·移

- 1. 应用领域（Web→Java/C#；系统→C；AI→LISP）
- 2. 用户要求
- 3. 现有工具环境
- 4. 开发环境成本
- 5. 程序员水平
- 6. 可移植性

IDE 组成

- 最小：编辑器 + 编译器 + 调试器
- 7 大功能：项目管理 / 编辑提示 / 文本编辑 / 跟踪调试 / 生成可执行文件 / 插件 / 屏幕管理
- 常见：VS、Eclipse、NetBeans、Delphi、JBuilder

结构化程序设计

- **核心**：自顶向下，逐步求精
- **三基本结构**：顺序、选择、循环（重复）
- **结构形状**：树状
- **原则**：一入口一出口；复杂结构＝基本结构嵌套
- 2009 单选 5 答案：D

面向对象程序设计

- **核心**：类的实现
- **特征**：对象、消息、类、继承
- **类的关系**：关联、聚合/组合、继承、依赖
- **类实现 7 项**：软件库、复用、断言、调试、内建错误处理、用户定义错误处理、多重实现
- **复用 2 法**：原封不动 / 进化性（继承）
- **断言 2 类**：先决条件 / 后置条件

程序设计风格 5 方面

1. 源程序文档化
2. 数据说明
3. 语句结构
4. 输入/输出
5. 错误处理

标准：使程序易于阅读和理解

源程序文档化·3 方面

1. 标识符命名
2. 程序注释（序言性＋功能性）
3. 源程序布局

2023 填空 6 答案：序言性注释

标识符命名 5 条

1. 英文全称反映含义（firstName）
2. 用专业术语（customer）
3. 注意大小写（Java：包小写、类首大写、变量驼峰、常量全大写）
4. 不宜过长（避免 thisstudentmanagementclass）
5. 避免轻微差异（SqlServer vs SQLServer）

2018 单选 10 答案：B（mailAddress）

★ 序言性 vs 功能性注释

	序言性	功能性
位置	模块开头	程序体内
粒度	模块级	程序段级
含"语句功能"?	否	是
含开发简历?	是	否
修改时	更新开发简历	改代码同时改注释

2009 单选 6 答案：B（语句的功能）

序言性注释 7 内容

1. 程序标题（模块名）
2. 功能和目的
3. 主要算法（option）
4. 接口说明
5. 数据描述
6. 模块位置
7. 开发简历（创建/修改者、时间、版本）

程序布局 8 条

1. 一行一语句
2. 缩进**2 字符**（不用 Tab）
3. 选择/循环体向右阶梯
4. 模块间空行分隔
5. 大括号独占一行
6. 结束括号缩进与开始一致
7. 不用无意义小括号
8. 小括号内侧**无空格**

数据说明 3 风格

1. 次序规范：常量→复杂类型→简单变量（整→实→字符→逻辑）→数组
2. 多变量同语句→按字母顺序
3. 复杂数据结构加注释

语句结构·核心

- **★ 清晰第一，效率第二**
- 一行一语句＋移行
- 首先正确，然后速度
- 用括号明确运算顺序
- 避免过多循环/条件嵌套
- 用伪码描述再翻译
- 模块功能单一
- 信息隐蔽
- 不修补坏程序，重写

OO 补充：方法≤50 行；少参数；少继承层数；少运算符重载

输入/输出 9 条

1. 输入数据**检验**
2. 检查组合合理性
3. 步骤简单
4. 允许自由格式
5. 允许默认值
6. 用**结束标志**而非指定数目
7. 交互式用**提示符**
8. 格式与语句要求一致
9. 输出**加注解**、设计报表

错误处理 5 法

1. 返回错误代码
2. 调用错误处理函数
3. 显示错误信息
4. 记录日志
5. 退出程序（高安全场景）

两类技术：避开错误（不让差错潜入） / 容错（影响最小化）

★ 真题速记

题	答
2009 单选5 三基本结构	D 顺序·选择·重复
2009 单选6 序言性不应有	B 语句的功能
2010 判断8 含单元测试	√
2018 单选10 规范命名	B mailAddress
2023 填空6 文档化	序言性注释

易混点提醒

- "重复"="循环"，三基本结构＝顺序·选择·循环（重复）
- 序言性注释是**模块级**，不解释语句
- 功能性注释描述**一段程序**，不是每个语句
- "清晰第一，效率第二"——除非对效率有特殊要求
- 缩进用**空格**，不用 Tab
- 广义实现包含**详细设计**；狭义不包含
- 类实现优先考虑**复用**（抽象类→具体类）

开放题得分关键词

- **程序设计风格 5 方面**：源程序文档化、数据说明、语句结构、I/O、错误处理
- **源程序文档化 3 方面**：标识符命名、程序注释、源程序布局
- **选语言 6 因素**：应用领域、用户要求、现有工具、成本、程序员水平、可移植性
- **实现任务 4 步**：选语言、选 IDE、算法设计、编码实现
- **结构化**：自顶向下·逐步求精；树状；顺序·选择·循环
- **OOP 核心**：类的实现；对象＋消息＋类＋继承

注释行数量

正规程序文本中，注释行数量占整个源程序的**1/3 到 1/2，甚至更多。**